

Proximal Policy Optimization (PPO)

Pietro Cestola

1 Introduzione

Proximal Policy Optimization (PPO) è un algoritmo di *reinforcement learning* appartenente alla famiglia dei policy gradient methods. Introdotto da Schulman et al. (2017), PPO combina semplicità di implementazione, stabilità numerica e buone prestazioni. Rispetto ad approcci come Trust Region Policy Optimization (TRPO), PPO evita vincoli di ottimizzazione complessi e introduce un obiettivo *clipped* che limita variazioni troppo grandi nella policy tra aggiornamenti successivi. Per queste ragioni è oggi uno standard de facto in moltissimi campi.

2 Richiami di Reinforcement Learning

Definizione 1 (Processo decisionale di Markov). Un processo decisionale di Markov (MDP) è una tupla

$$M = (S, A, P, r, \gamma),$$

dove:

- S è l'insieme di tutti gli **stati** dell'ambiente;
- A è l'insieme di tutte le **azioni** possibili;
- $P(s'|s, a)$ è la funzione di **transizione**, ossia la probabilità che, trovandosi nello stato s ed eseguendo l'azione a , l'ambiente transiti nello stato s' ;
- $r(s, a)$ è la funzione di **ricompensa attesa**, cioè il valore medio della ricompensa immediata ottenuta eseguendo a in s ;
- $\gamma \in [0, 1]$ è il **fattore di sconto**, che determina l'importanza relativa delle ricompense future.

Definizione 2 (Policy). Una **policy** è una distribuzione di probabilità sulle azioni condizionata dallo stato, formalmente una mappa

$$\pi : S \times A \rightarrow [0, 1], \quad (s, a) \mapsto \pi(a|s),$$

tale che, per ogni stato $s \in S$, la funzione $a \mapsto \pi(a|s)$ sia una distribuzione di probabilità sulle azioni possibili A , ossia,

$$\sum_{a \in A} \pi(a|s) = 1.$$

Definizione 3 (Dinamica). Dato $M = (S, A, P, r, \gamma)$, una policy π ed uno stato iniziale s_0 estratto tramite una distribuzione ρ su S , si definisce **dinamica** la legge probabilistica che descrive l'evoluzione agente-ambiente:

- al passo t , l'agente osserva lo stato $s_t \in S$;
- seleziona un'azione $a_t \in A$ campionando da $\pi(\cdot|s_t)$;
- riceve la ricompensa $r_{t+1} = r(s_t, a_t)$;
- l'ambiente transita in un nuovo stato s_{t+1} secondo $P(\cdot|s_t, a_t)$.

Una **traiettoria** τ è una singola realizzazione della dinamica:

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots),$$

la probabilità di una traiettoria finita di lunghezza T è:

$$\mathbb{P}(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) P(s_{t+1}|s_t, a_t).$$

Definizione 4 (Ricompensa totale scontata). Dato $M = (S, A, P, r, \gamma)$ e una traiettoria $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$ generata secondo la dinamica associata ad una policy π , la **ricompensa totale scontata** a partire dal passo t è definita come

$$R_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1},$$

ossia la somma, scontata nel tempo, di tutte le ricompense future ricevute dopo il tempo t . Per episodi finiti di lunghezza T , la somma è intesa fino al termine dell'episodio:

$$R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}.$$

Definizione 5 (Funzioni valore e vantaggio). Dato $M = (S, A, P, r, \gamma)$ ed una policy π si definiscono le funzioni di valore:

$$V^\pi(s) = \mathbb{E}_\pi[R_t | s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[R_t | s_t = s, a_t = a],$$

e il **vantaggio**

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

3 Richiami di Policy Gradient Methods

I *policy gradient methods* sono una famiglia di algoritmi di reinforcement learning che ottimizzano direttamente la policy dell'agente per massimizzare il rendimento atteso. L'idea di base è trattare la policy come un oggetto variabile e cercare, tramite l'ottimizzazione, i parametri che la rendono più performante nell'MDP dato.

Per fare ciò, è necessario poter modificare la policy in modo continuo e calcolare come piccole variazioni dei suoi parametri influenzano il rendimento. Si introduce quindi il concetto di **policy parametrizzata**, che consente di usare metodi di ottimizzazione basati sul gradiente.

Definizione 6 (Policy parametrizzata). Sia $\Theta \subseteq \mathbb{R}^d$ uno spazio di parametri. Una policy parametrizzata è una funzione

$$\pi : S \times A \times \Theta \rightarrow [0, 1], \quad (s, a, \theta) \mapsto \pi(a|s; \theta),$$

differenziabile rispetto a θ , tale che per ogni $\theta \in \Theta$ e stato $s \in S$ la mappa $a \mapsto \pi(a|s; \theta)$ è una distribuzione di probabilità sulle azioni, ossia,

$$\sum_{a \in A} \pi(a|s; \theta) = 1.$$

In pratica, $\pi(a|s; \theta)$ è spesso implementata come una **rete neurale** che prende in ingresso lo stato e restituisce i parametri di una distribuzione sulle azioni.

Definizione 7 (Rendimento atteso). Dato M e una policy parametrizzata π_θ , il rendimento atteso è definito come

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R_0] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right].$$

L'obiettivo è massimizzare il ritorno atteso $J(\theta)$. I *policy gradient methods* si caratterizzano per l'uso del gradiente del rendimento atteso rispetto ai parametri della policy $\nabla_\theta J(\theta)$, detto **policy gradient**, per guidare l'aggiornamento. Un esempio concreto è l'aggiornamento tramite stochastic gradient ascent (SGA), che utilizza il gradiente per incrementare iterativamente i parametri nella direzione di massima crescita del ritorno atteso:

$$\theta_{n+1} = \theta_n + \eta \nabla_\theta J(\theta_n).$$

Per usare concretamente questi algoritmi serve avere un'espressione del gradiente della policy calcolabile numericamente. Questo richiede due fasi: prima si trova l'espressione analitica del policy gradient, che come vedremo di seguito ha la forma di un valore atteso, poi tale valore viene sostituito con una stima campionaria ottenuta dai dati raccolti in un numero finito di interazioni agente-ambiente. Lo stimatore così ottenuto $\hat{g}(\theta)$ viene infine utilizzato per aggiornare i parametri secondo lo schema scelto, ad esempio SGA:

$$\theta_{n+1} = \theta_n + \eta \hat{g}(\theta_n).$$

Questa sostituzione introduce varianza e rende necessarie strategie dedicate per controllarla e stabilizzare l'apprendimento, il **PPO** è stato progettato proprio con questo scopo.

Teorema 1 (Espressione analitica del policy gradient). L'espressione analitica del policy gradient è

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_0(\tau) \right]$$

Dimostrazione: vedi Appendice A.1.

Poiché l'espressione analitica del policy gradient è un valore atteso, possiamo stimarlo con una media campionaria. Se raccogliamo un insieme di traiettorie $\mathcal{D} = \{\tau_i\}_{i=1, \dots, N}$, dove ogni traiettoria è generata lasciando l'agente agire nell'ambiente seguendo la policy π_{θ} , otteniamo la seguente stima del gradiente:

$$\hat{g}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_0(\tau).$$

Prendendo un passo lungo il gradiente nella forma del Teorema 1, si incrementano le probabilità delle azioni in proporzione a $R_0(\tau)$, ossia alla somma complessiva delle ricompense osservate sulla traiettoria. Tale formulazione, però, non risulta del tutto appropriata: un agente dovrebbe rinforzare le proprie azioni esclusivamente in base alle loro conseguenze. Le ricompense accumulate prima dell'esecuzione di un'azione non forniscono informazioni utili sulla bontà di quell'azione; solo le ricompense successive ne determinano il valore. Questa intuizione è confermata dai seguenti risultati: il policy gradient può infatti essere riscritto in una forma in cui le azioni sono rinforzate unicamente in funzione delle ricompense ottenute dopo la loro esecuzione.

Teorema 2 (EGLP Lemma). Sia P_{θ} una distribuzione di probabilità parametrizzata su uno spazio Ω . Allora:

$$\mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)] = 0.$$

Dimostrazione: vedi Appendice A.2.

Corollario 1. Il policy gradient può essere scritto nel seguente modo

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t(\tau) \right]$$

Dimostrazione: Segue direttamente dal Teorema 2.

Tale riformulazione risulta vantaggiosa per una ragione fondamentale: nei policy gradient methods un problema critico è l'elevata varianza delle stime campionarie del gradiente, che impone di raccogliere molte traiettorie per ottenere una stima accurata. Nella formula originale erano presenti termini proporzionali a ricompense passate, aventi media nulla ma varianza non nulla, che contribuiscono soltanto a introdurre rumore nelle stime. Eliminandoli si riduce la varianza e, di conseguenza, diminuisce il numero di traiettorie necessarie per un apprendimento efficace.

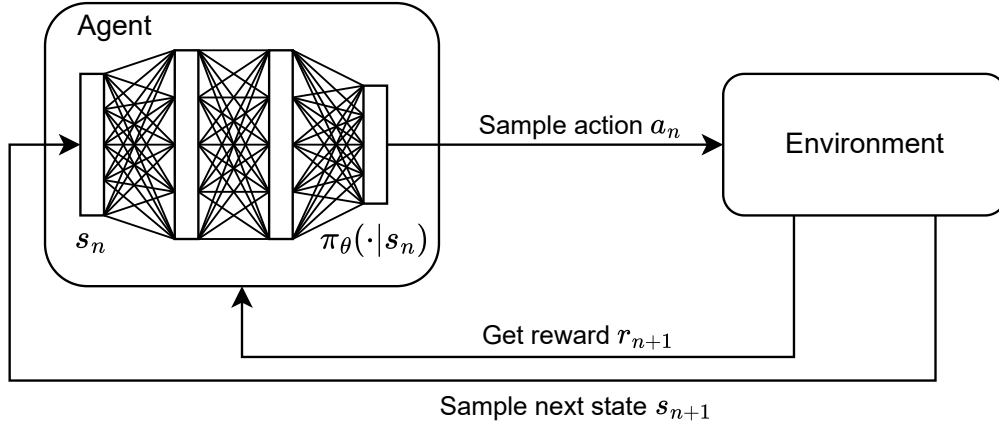


Figura 1: Schema ambiente-agente con multilayer perceptron come policy parametrizzata.

4 Proximal Policy Optimization

L'aggiornamento dei parametri con il gradiente stimato può produrre variazioni troppo ampie della policy, causando instabilità e degradando rapidamente le prestazioni. Per affrontare questo problema, PPO parte dai seguenti risultati.

4.1 Risultati Preliminari

Proposizione 1. Sia $b : S \rightarrow \mathbb{R}$ una funzione dipendente dai soli stati detta *bias*. Allora

$$\mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = 0$$

Dimostrazione: Segue direttamente dal Teorema 2.

Nota: Questo ci permette di aggiungere o sottrarre una generica funzione bias dipendente dai soli stati dall'espressione del policy gradient, senza modificarne il valore.

Teorema 3. Sia $M = (S, A, P, r, \gamma)$ un MDP, sia $\pi_\theta(a|s)$ una policy parametrizzata, e sia q una distribuzione iniziale sugli stati S . Allora il gradiente del rendimento atteso è dato da

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) Q^{\pi_\theta}(s_t, a_t) \right],$$

Dimostrazione: vedi Appendice A.3.

Corollario 2. Sia $M = (S, A, P, r, \gamma)$ un MDP, sia $\pi_\theta(a|s)$ una policy parametrizzata, e sia q una distribuzione iniziale sugli stati S . Allora il gradiente del rendimento atteso è dato da

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) A^{\pi_\theta}(s_t, a_t) \right],$$

Dimostrazione: Segue immediatamente dal Teorema 3, dalla Proposizione 1 scegliendo $b(s) = V^{\pi_\theta}(s)$ e dalla definizione di vantaggio.

4.2 Algoritmo PPO

Definizione 8 (Rapporto di probabilità). Il **rapporto di probabilità** misura come la nuova policy valuta un'azione rispetto alla policy vecchia:

$$\rho(s, a; \theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_n}(a|s)}.$$

Definizione 9 (Surrogato del policy gradient). Il **surrogato** usato per aggiornare i parametri con i dati raccolti sotto π_{θ_n} è

$$\mathcal{L}^{PG}(\theta, \theta_n) = \mathbb{E}_{\tau \sim \pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) A^{\pi_{\theta_n}}(s_t, a_t) \right].$$

Teorema 4. Il gradiente del surrogato calcolato nel punto θ_n corrisponde al policy gradient:

$$\nabla_\theta \mathcal{L}^{PG}(\theta_n, \theta_n) = \nabla_\theta J(\theta_n).$$

Dimostrazione: vedi Appendice A.4.

Il Teorema 4 costituisce il fondamento teorico su cui si basa il PPO: nel punto θ_n il gradiente del surrogato $\mathcal{L}^{PG}$ coincide con il gradiente del rendimento atteso $J(\theta)$. L'introduzione di $\mathcal{L}^{PG}(\theta, \theta_n)$ è funzionale all'ottimizzazione con dati finiti:

- **Riutilizzo on-policy.** Poiché l'attesa è calcolata rispetto a π_{θ_n} , gli stessi dati raccolti con π_{θ_n} forniscono stime di $\nabla_\theta \mathcal{L}^{PG}(\theta, \theta_n)$ per θ in un intorno di θ_n , grazie al ri-pesamento per importanza $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_n}(a_t|s_t)$. Con la sola espressione di $\nabla_\theta J(\theta)$ si ottiene una direzione corretta in θ_n , ma ulteriori micro-aggiornamenti richiederebbero nuove traiettorie per restare on-policy, con costo di campionamento elevato.
- **Mini-Batch.** $\mathcal{L}^{PG}$ fornisce una funzione scalare differenziabile su cui applicare più epoche di ascesa del gradiente stocastico usando mini-batch estratti dai dati, mantenendo al primo ordine la stessa direzione di miglioramento di J nel punto θ_n , in virtù del Teorema 4.

L'efficacia di $\mathcal{L}^{PG}$ è dunque legata alla possibilità di effettuare più aggiornamenti per ogni raccolta dati, aumentando l'efficienza campionaria, pur rimanendo coerenti con il gradiente vero nel punto corrente. Il limite di questa costruzione è che, allontanandosi da θ_n , i pesi $\rho_t(\theta)$ possono crescere e amplificare varianza e instabilità. Per questo motivo, l'algoritmo utilizza una versione *clipped* del surrogato:

$$\mathcal{L}^{PPO}(\theta, \theta_n) = \mathbb{E}_{\tau \sim \pi_{\theta_n}} \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

che limita implicitamente la deviazione quando $|\rho_t(\theta) - 1|$ aumenta.

Teorema 5. Nel punto θ_n vale l'uguaglianza

$$\nabla_\theta \mathcal{L}^{PPO}(\theta_n, \theta_n) = \nabla_\theta \mathcal{L}^{PG}(\theta_n, \theta_n).$$

Dimostrazione: segue direttamente dalla definizione di clip.

Questo risultato garantisce che la funzione obiettivo troncata preservi la correttezza teorica del policy gradient nel punto attuale, ma limiti la variazione della policy quando ci si allontana troppo. L'algoritmo consiste infine nel seguente schema iterativo:

Algoritmo PPO

- 1) Si parte con una policy casuale π_{θ_n} dove $n = 0$.
- 2) Si raccoglie un insieme di traiettorie sotto π_{θ_n} e si calcola una stima di $A^{\pi_{\theta_n}}(s_t, a_t)$.
- 3) Si massimizza $\mathcal{L}^{\text{PPO}}$ usando ottimizzatori a gradiente (ad esempio Adam) per un numero limitato di epoche, suddividendo i dati in mini-batch.
- 4) Dopo alcune iterazioni di ascesa del gradiente su $\mathcal{L}^{\text{PPO}}$, si ottiene un nuovo parametro θ_{n+1} con il quale si torna al passo 2.

In sintesi: rispetto a un policy gradient on-policy “vanilla”, in cui si esegue *un solo* aggiornamento per ogni batch di traiettorie e poi si deve simulare di nuovo per restare on-policy, PPO consente di **riutilizzare lo stesso batch per più aggiornamenti** (più epoche/mini-batch). Questo è possibile grazie al surrogato con rapporto di probabilità e al *clipping*: nel punto θ_n la direzione di aggiornamento coincide con quella del gradiente vero, e quando ci si allontana il clipping controlla lo scostamento della policy. Il risultato è una **maggiore efficienza campionaria**: a parità di numero di aggiornamenti, servono meno simulazioni.

A Appendice

A.1 Dimostrazioni del Teorema 1

Dimostrazione. Sappiamo già che la probabilità di una traiettoria τ è data da

$$\mathbb{P}_\theta(\tau) = \rho(s_0) \prod_{t \geq 0} \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t).$$

Poiché ρ e P non dipendono da θ segue che,

$$\nabla_\theta \log \mathbb{P}_\theta(\tau) = \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t).$$

Usando questo fatto ed il *trucco del logaritmo* ($\nabla f = f \nabla \log f$) otteniamo la tesi:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \sum_{\tau} \mathbb{P}_\theta(\tau) R_0(\tau) \\ &= \sum_{\tau} \nabla_\theta \mathbb{P}_\theta(\tau) R_0(\tau) \\ &= \sum_{\tau} \mathbb{P}_\theta(\tau) \nabla_\theta \log \mathbb{P}_\theta(\tau) R_0(\tau) \\ &= \sum_{\tau} \mathbb{P}_\theta(\tau) \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) R_0(\tau) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) R_0(\tau) \right]. \end{aligned}$$

□

A.2 Dimostrazione dell'Expected Grad-Log Prob Lemma

Dimostrazione. Denotiamo con $\mathcal{X}$ lo spazio misurabile su cui è definita la distribuzione. Osserviamo anzitutto che

$$\nabla_{\theta} \int_{\mathcal{X}} P_{\theta}(x) dx = \nabla_{\theta} 1 = 0,$$

da questo fatto e dal *trucco del logaritmo* segue la tesi

$$0 = \nabla_{\theta} \int_{\mathcal{X}} P_{\theta}(x) dx = \int_{\mathcal{X}} \nabla_{\theta} P_{\theta}(x) dx = \int_{\mathcal{X}} P_{\theta}(x) \nabla_{\theta} \log P_{\theta}(x) dx = \mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)].$$

□

A.3 Dimostrazione del Teorema 3

Dimostrazione. Definiamo $\tau_{:t} = (s_0, a_0, \dots, s_t, a_t)$ come la traiettoria fino al tempo t e con τ_t la rimanente parte dopo di essa. Dalla legge delle aspettative iterate e dalla proprietà di Markov segue che

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t(\tau) \right] \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t(\tau)] \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau_{:t} \sim \pi_{\theta}} [\mathbb{E}_{\tau_t: \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t(\tau) | \tau_{:t}]] \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau_{:t} \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{\tau_t: \sim \pi_{\theta}} [R_t(\tau) | \tau_{:t}]] \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau_{:t} \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{\tau_t: \sim \pi_{\theta}} [R_t(\tau) | s_t, a_t]] \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau_{:t} \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t)], \end{aligned}$$

che è la tesi. □

A.4 Dimostrazione dell'Equivalenza del gradiente nel punto θ_n

Dimostrazione. Per definizione

$$\mathcal{L}^{PG}(\theta, \theta_n) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) A^{\pi_{\theta_n}}(s_t, a_t) \right], \quad \rho(s_t, a_t; \theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_n}(a_t | s_t)}.$$

Derivando:

$$\nabla_{\theta} \rho(s_t, a_t; \theta) = \rho(s_t, a_t; \theta) \nabla_{\theta} \log \rho(s_t, a_t; \theta) = \rho(s_t, a_t; \theta) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Quindi

$$\nabla_{\theta} \mathcal{L}^{PG}(\theta, \theta_n) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta_n}}(s_t, a_t) \right].$$

Valutando in $\theta = \theta_n$ si ha $\rho(s_t, a_t; \theta_n) = 1$, quindi

$$\nabla_{\theta} \mathcal{L}^{PG}(\theta_n, \theta_n) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta_n}(a_t | s_t) A^{\pi_{\theta_n}}(s_t, a_t) \right] = \nabla_{\theta} J(\theta_n).$$

□